

开源软件基础

Python 基础

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 16, 2017



Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - Numbers
 - Strings
 - Lists
- ④ 函数
- ⑤ 递归

为什么选择 Python

- 流行
- 简单 (?)
- 丰富的生态系统
- 交互界面友好

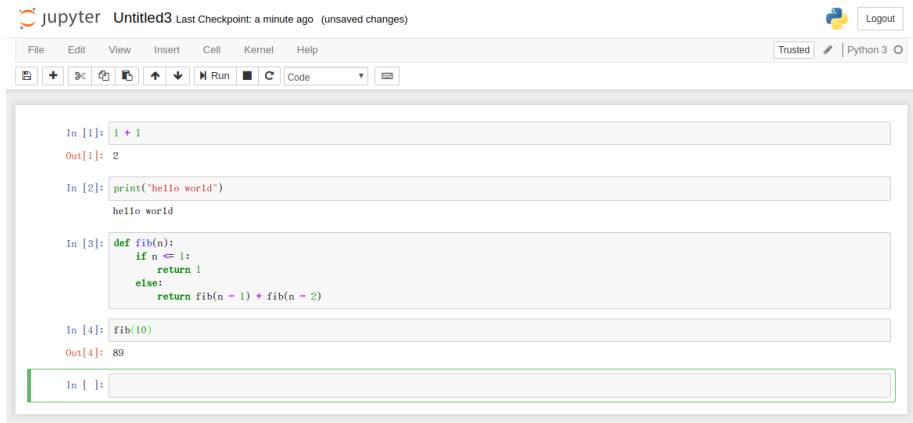
Python 版本

- Python 存在两个用途广泛的版本：2.x 和 3.x
- 两者不完全兼容
- Python 2.x 分支版本在 2020 年将停止更新
- Python 3 对 UTF-8 支持更好
- 很多公司/开源社区仍默认选择 Python 2

Python 发行版

- vanilla Python
- Anaconda
- Python (x,y)
- Active Python

Jupyter Notebook



The screenshot shows a Jupyter Notebook interface with the following elements:

- Header:** "jupyter Untitled3 Last Checkpoint: a minute ago (unsaved changes)" and a "Logout" button.
- Menu Bar:** File, Edit, View, Insert, Cell, Kernel, Help.
- Toolbar:** Includes buttons for saving, undo, redo, copy, paste, run, and a dropdown menu set to "Code".
- Code Cells:**
 - Cell 1:** `In [1]: 1 + 1` with output `Out[1]: 2`.
 - Cell 2:** `In [2]: print("hello world")` with output `hello world`.
 - Cell 3:** `In [3]: def fib(n):
if n <= 1:
return 1
else:
return fib(n - 1) + fib(n - 2)`
 - Cell 4:** `In [4]: fib(10)` with output `Out[4]: 89`.
 - Cell 5:** An empty input prompt `In [ ]:` with a green border.

本学期将涉及的 Python 知识

知识点

- 流程控制（结构化）
- 面向对象（皮毛）
- 数据获取
- 数据存储
- 数据可视化
- Web 开发（皮毛）

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - Numbers
 - Strings
 - Lists
- 4 函数
- 5 递归

第一个 Python 程序

```
$ipython3
Python 3.5.2+ (default, Sep 22 2016, 12:18:14)
Type ``copyright``, ``credits`` or ``license`` for more information.

IPython 5.1.0 -- An enhanced Interactive Python.
?                -> Introduction and overview of IPython's features.
%quickref        -> Quick reference.
help             -> Python's own help system.
object?         -> Details about 'object', use 'object??' for extra details.

In [1]:
```

第一个 Python 程序

```
In [1]: print("Hello world")  
Hello world
```

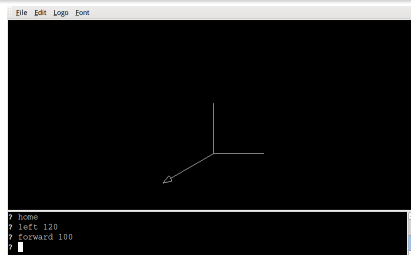
```
In [2]: print(1 + 2)  
3
```

```
In [3]:
```

Python Turtle

Logo

Python Turtle 受 Logo 语言启发而来。



Python Turtle

```
to tree :len :n
  if :n < 1 [stop]
  forward :len
  left 45
  tree (:len / 2) (:n - 1)
  right 90
  tree (:len / 2) (:n - 1)
  left 45
  back :len
end
```

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - Numbers
 - Strings
 - Lists
- 4 函数
- 5 递归

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - Numbers
 - Strings
 - Lists
- 4 函数
- 5 递归

Numbers

The interpreter acts as a simple calculator: you can type an expression at it and it will write the value. Expression syntax is straightforward: the operators `+`, `-`, `*` and `/` work just like in most other languages (for example, Pascal or C); parentheses `()` can be used for grouping. For example:

Numbers

```
>>> 2 + 2
```

```
4
```

```
>>> 50 - 5*6
```

```
20
```

```
>>> (50 - 5*6) / 4
```

```
5.0
```

```
>>> 8 / 5 # division always returns a floating point
```

```
1.6
```

Numbers

The integer numbers (e.g. 2, 4, 20) have type `int`, the ones with a fractional part (e.g. 5.0, 1.6) have type `float`. We will see more about numeric types later in the tutorial.

Numbers

Division (/) always returns a float. To do floor division and get an integer result (discarding any fractional result) you can use the // operator; to calculate the remainder you can use %:

Numbers

```
>>> 17 / 3 # classic division returns a float
5.666666666666667
>>>
>>> 17 // 3 # floor division discards the fractional
5
>>> 17 % 3 # the % operator returns the remainder of
2
>>> 5 * 3 + 2 # result * divisor + remainder
17
```

Numbers

With Python, it is possible to use the `**` operator to calculate powers [1]:

Numbers

```
>>> 5 ** 2 # 5 squared
```

```
25
```

```
>>> 2 ** 7 # 2 to the power of 7
```

```
128
```

Numbers

The equal sign (=) is used to assign a value to a variable. Afterwards, no result is displayed before the next interactive prompt:

Numbers

```
>>> width = 20  
>>> height = 5 * 9  
>>> width * height  
900
```

Numbers

If a variable is not "defined" (assigned a value), trying to use it will give you an error:

Numbers

```
>>> n # try to access an undefined variable
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

Numbers

There is full support for floating point; operators with mixed type operands convert the integer operand to floating point:

Numbers

```
>>> 4 * 3.75 - 1  
14.0
```

Numbers

In interactive mode, the last printed expression is assigned to the variable `_`. This means that when you are using Python as a desk calculator, it is somewhat easier to continue calculations, for example:

Numbers

```
>>> tax = 12.5 / 100
>>> price = 100.50
>>> price * tax
12.5625
>>> price + _
113.0625
>>> round(_, 2)
113.06
```

Numbers

This variable should be treated as read-only by the user. Don't explicitly assign a value to it — you would create an independent local variable with the same name masking the built-in variable with its magic behavior.

Numbers

In addition to `int` and `float`, Python supports other types of numbers, such as `decimal.Decimal` and `fractions.Fraction`. Python also has built-in support for complex numbers `<typesnumeric>`, and uses the `j` or `J` suffix to indicate the imaginary part (e.g. `3+5j`).

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - Numbers
 - **Strings**
 - Lists
- ④ 函数
- ⑤ 递归

Strings

Besides numbers, Python can also manipulate strings, which can be expressed in several ways. They can be enclosed in single quotes ('...') or double quotes ("...") with the same result [2]. \ can be used to escape quotes:

Strings

```
>>> 'spam eggs'    # single quotes
'spam eggs'
>>> 'doesn\'t'      # use \' to escape the single quote...
"doesn't"
>>> "doesn't"       # ...or use double quotes instead
"doesn't"
>>> '"Yes," he said.'
'"Yes," he said.'
>>> "\"Yes,\" he said."
'"Yes," he said.'
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
```

Strings

In the interactive interpreter, the output string is enclosed in quotes and special characters are escaped with backslashes. While this might sometimes look different from the input (the enclosing quotes could change), the two strings are equivalent. The string is enclosed in double quotes if the string contains a single quote and no double quotes, otherwise it is enclosed in single quotes. The print function produces a more readable output, by omitting the enclosing quotes and by printing escaped and special characters:

Strings

```
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
>>> print('"Isn\'t," she said.')
"Isn't," she said.
>>> s = 'First line.\nSecond line.' # \n means newline
>>> s # without print(), \n is included in the output
'First line.\nSecond line.'
>>> print(s) # with print(), \n produces a new line
First line.
Second line.
```

Strings

If you don't want characters prefaced by `\` to be interpreted as special characters, you can use raw strings by adding an `r` before the first quote:

Strings

```
>>> print('C:\some\name')    # here \n means newline!
C:\some
ame
>>> print(r'C:\some\name')    # note the r before the qu
C:\some\name
```

Strings

String literals can span multiple lines. One way is using triple-quotes: `"""..."""` or `'''...'''`. End of lines are automatically included in the string, but it's possible to prevent this by adding a `\` at the end of the line. The following example:

Strings

```
print("""\
Usage: thingy [OPTIONS]
    -h                Display this usage mess
    -H hostname       Hostname to connect to
""")
```

Display this usage mess
Hostname to connect to

Strings

produces the following output (note that the initial newline is not included):

Strings

```
Usage: thingy [OPTIONS]
```

```
-h
```

Display this usage mess

```
-H hostname
```

Hostname to connect to

Strings

Strings can be concatenated (glued together) with the `+` operator, and repeated with `*`:

Strings

```
>>> # 3 times 'un', followed by 'ium'
>>> 3 * 'un' + 'ium'
'ununinium'
```

Strings

Two or more string literals (i.e. the ones enclosed between quotes) next to each other are automatically concatenated.

Strings

```
>>> 'Py' 'thon'  
'Python'
```

Strings

This only works with two literals though, not with variables or expressions:

Strings

```
>>> prefix = 'Py'
>>> prefix 'thon'    # can't concatenate a variable and
...
SyntaxError: invalid syntax
>>> ('un' * 3) 'ium'
...
SyntaxError: invalid syntax
```

Strings

If you want to concatenate variables or a variable and a literal, use `+`:

Strings

```
>>> prefix + 'thon'  
'Python'
```

Strings

This feature is particularly useful when you want to break long strings:

Strings

```
>>> text = ('Put several strings within parentheses '  
...        'to have them joined together.')
```

>>> text

'Put several strings within parentheses to have them j

Strings

Strings can be indexed (subscripted), with the first character having index 0. There is no separate character type; a character is simply a string of size one:

Strings

```
>>> word = 'Python'
>>> word[0]    # character in position 0
'P'
>>> word[5]    # character in position 5
'n'
```

Strings

Indices may also be negative numbers, to start counting from the right:

Strings

```
>>> word[-1]    # last character
'n'
>>> word[-2]    # second-last character
'o'
>>> word[-6]
'p'
```

Strings

Note that since -0 is the same as 0, negative indices start from -1.

Strings

In addition to indexing, slicing is also supported. While indexing is used to obtain individual characters, slicing allows you to obtain substring:

Strings

```
>>> word[0:2]    # characters from position 0 (included)
'Py'
>>> word[2:5]    # characters from position 2 (included)
'tho'
```

Strings

Note how the start is always included, and the end always excluded.
This makes sure that $s[:i] + s[i:]$ is always equal to s :

Strings

```
>>> word[:2] + word[2:]  
'Python'  
>>> word[:4] + word[4:]  
'Python'
```

Strings

Slice indices have useful defaults; an omitted first index defaults to zero, an omitted second index defaults to the size of the string being sliced.

Strings

```
>>> word[:2]    # character from the beginning to posit
'Py'
>>> word[4:]    # characters from position 4 (included)
'on'
>>> word[-2:]   # characters from the second-last (incl
'on'
```

Strings

One way to remember how slices work is to think of the indices as pointing between characters, with the left edge of the first character numbered 0. Then the right edge of the last character of a string of n characters has index n , for example:

Strings

```

+---+---+---+---+---+---+
| P | y | t | h | o | n |
+---+---+---+---+---+---+
0   1   2   3   4   5   6
-6  -5  -4  -3  -2  -1

```

Strings

The first row of numbers gives the position of the indices 0...6 in the string; the second row gives the corresponding negative indices. The slice from i to j consists of all characters between the edges labeled i and j , respectively.

Strings

For non-negative indices, the length of a slice is the difference of the indices, if both are within bounds. For example, the length of `word[1:3]` is 2.

Strings

Attempting to use an index that is too large will result in an error:

Strings

```
>>> word[42] # the word only has 6 characters
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

Strings

However, out of range slice indexes are handled gracefully when used for slicing:

Strings

```
>>> word[4:42]  
'on'  
>>> word[42:]  
,,
```

Strings

Python strings cannot be changed — they are immutable. Therefore, assigning to an indexed position in the string results in an error:

Strings

```
>>> word[0] = 'J'
```

```
...
```

```
TypeError: 'str' object does not support item assignment
```

```
>>> word[2:] = 'py'
```

```
...
```

```
TypeError: 'str' object does not support item assignment
```

Strings

If you need a different string, you should create a new one:

Strings

```
>>> 'J' + word[1:]  
'Jython'  
>>> word[:2] + 'py'  
'Pypy'
```

Strings

The built-in function `len` returns the length of a string:

Strings

```
>>> s = 'supercalifragilisticexpialidocious'  
>>> len(s)  
34
```

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - Numbers
 - Strings
 - **Lists**
- 4 函数
- 5 递归

Lists

Python knows a number of compound data types, used to group together other values. The most versatile is the list, which can be written as a list of comma-separated values (items) between square brackets. Lists might contain items of different types, but usually the items all have the same type.

Lists

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]
```

Lists

Like strings (and all other built-in sequence type), lists can be indexed and sliced:

Lists

```
>>> squares[0]    # indexing returns the item
1
>>> squares[-1]
25
>>> squares[-3:]  # slicing returns a new list
[9, 16, 25]
```

Lists

All slice operations return a new list containing the requested elements. This means that the following slice returns a new (shallow) copy of the list:

Lists

```
>>> squares[:]  
[1, 4, 9, 16, 25]
```

Lists

Lists also support operations like concatenation:

Lists

```
>>> squares + [36, 49, 64, 81, 100]  
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Lists

Unlike strings, which are immutable, lists are a mutable type, i.e. it is possible to change their content:

Lists

```
>>> cubes = [1, 8, 27, 65, 125] # something's wrong h
>>> 4 ** 3 # the cube of 4 is 64, not 65!
64
>>> cubes[3] = 64 # replace the wrong value
>>> cubes
[1, 8, 27, 64, 125]
```

Lists

You can also add new items at the end of the list, by using the `list.append` method (we will see more about methods later):

Lists

```
>>> cubes.append(216)    # add the cube of 6
>>> cubes.append(7 ** 3)  # and the cube of 7
>>> cubes
[1, 8, 27, 64, 125, 216, 343]
```

Lists

Assignment to slices is also possible, and this can even change the size of the list or clear it entirely:

Lists

```
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> # replace some values
>>> letters[2:5] = ['C', 'D', 'E']
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> # now remove them
>>> letters[2:5] = []
>>> letters
['a', 'b', 'f', 'g']
>>> # clear the list by replacing all the elements with
>>> letters[:] = []
```

Lists

The built-in function `len` also applies to lists:

Lists

```
>>> letters = ['a', 'b', 'c', 'd']  
>>> len(letters)  
4
```

Lists

It is possible to nest lists (create lists containing other lists), for example:

Lists

```
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> x = [a, n]
>>> x
[['a', 'b', 'c'], [1, 2, 3]]
>>> x[0]
['a', 'b', 'c']
>>> x[0][1]
'b'
```

First Steps Towards Programming

Of course, we can use Python for more complicated tasks than adding two and two together. For instance, we can write an initial sub-sequence of the Fibonacci series as follows:

First Steps Towards Programming

```
>>> # Fibonacci series:
... # the sum of two elements defines the next
... a, b = 0, 1
>>> while b < 10:
...     print(b)
...     a, b = b, a+b
...
1
1
2
3
5
8
```

First Steps Towards Programming

This example introduces several new features.

First Steps Towards Programming

The first line contains a multiple assignment: the variables `a` and `b` simultaneously get the new values 0 and 1. On the last line this is used again, demonstrating that the expressions on the right-hand side are all evaluated first before any of the assignments take place. The right-hand side expressions are evaluated from the left to the right.

First Steps Towards Programming

The while loop executes as long as the condition (here: $b < 10$) remains true. In Python, like in C, any non-zero integer value is true; zero is false. The condition may also be a string or list value, in fact any sequence; anything with a non-zero length is true, empty sequences are false. The test used in the example is a simple comparison. The standard comparison operators are written the same as in C: $<$ (less than), $>$ (greater than), $==$ (equal to), $<=$ (less than or equal to), $>=$ (greater than or equal to) and $!=$ (not equal to).

First Steps Towards Programming

The body of the loop is indented: indentation is Python's way of grouping statements. At the interactive prompt, you have to type a tab or space(s) for each indented line. In practice you will prepare more complicated input for Python with a text editor; all decent text editors have an auto-indent facility. When a compound statement is entered interactively, it must be followed by a blank line to indicate completion (since the parser cannot guess when you have typed the last line). Note that each line within a basic block must be indented by the same amount.

First Steps Towards Programming

The print function writes the value of the argument(s) it is given. It differs from just writing the expression you want to write (as we did earlier in the calculator examples) in the way it handles multiple arguments, floating point quantities, and strings. Strings are printed without quotes, and a space is inserted between items, so you can format things nicely, like this:

First Steps Towards Programming

```
>>> i = 256*256  
>>> print('The value of i is', i)  
The value of i is 65536
```

First Steps Towards Programming

The keyword argument `end` can be used to avoid the newline after the output, or end the output with a different string:

First Steps Towards Programming

```
>>> a, b = 0, 1
>>> while b < 1000:
...     print(b, end=', ')
...     a, b = b, a+b
...
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987,
```

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - Numbers
 - Strings
 - Lists
- ④ 函数
- ⑤ 递归

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - Numbers
 - Strings
 - Lists
- 4 函数
- 5 递归

递归

To understand recursion, first you have to understand recursion.

阶乘

```
def factorial(n):  
    if n <= 1:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

递归

With a little help from my friends.

– The Beatles

分形

分形 (Fractal), 又称碎形、残形, 通常被定义为「一个粗糙或零碎的几何形状, 可以分成数个部分, 且每一部分都 (至少近似地) 是整体缩小后的形状」, 即具有自相似的性质。¹

¹<https://zh.wikipedia.org/wiki/>

Turtle

```
import turtle

def tree(length,n):
    """ paints a branch of a tree """
    if length < (length/n):
        return
    turtle.forward(length)
    turtle.left(45)
    tree(length * 0.5,length/n)
    turtle.right(90)
    tree(length * 0.5,length/n)
    turtle.left(45)
    turtle.backward(length)
    return
```

边界条件
主干
左转分叉
左子树
右转分叉
右子树
转回
回到起点
返回



Turtle

